Simple Synthesizer

Katrina Macairan
College of Engineering
California State University, Long Beach
Long Beach, United States of America
Katrina.Macairan@student.csulb.edu

Kevin Phan
College of Engineering
California State University, Long Beach
Long Beach, United States of America
Kevin.Phan@student.csulb.edu

James Quon
College of Engineering
California State University, Long Beach
Long Beach, United States of America
James.Quon01@student.csulb.edu

***Abstract*—In this project, we aim to create a versatile and interactive synthesizer using the Nexys A7-100T FPGA board. The synthesizer will allow users to play individual notes, similar to a piano, with sound output coming through an external speaker. The system will be directed through a combination of external push buttons and onboard switches, with the switches allowing users to adjust the octave range for the playable notes. Through the combination of FPGA programming and hardware interfacing to create an interactive musical instrument, this project will open up possibilities for sound expression.**

Keywords— Synthesizer, Onboard, Counter, Buttons

I. INTRODUCTION

This FPGA digital synthesizer project will serve as a culmination of our digital logic design and verification knowledge thus far. Sequential and combinational circuits will be present in our implementation. 13 buttons will be interfaced to the Digilent Nexys A7-100T's GPIO ports to represent piano keys, starting from the C note and ascending by a half-step until the next C note (an octave). Two onboard switches will be used to change the octave that the physical "keyboard" encompasses. By default, and when no switches are flipped, the notes C3 to C4 are playable. When the rightmost switch is on, the range of the keyboard becomes C4 to C5. If the leftmost switch is on, the range of the keyboard becomes C5 to C6. Multiple notes can be played at a time. A speaker will be connected to the Nexys's mono audio output port. In addition, onboard LEDs will light when a corresponding key is pressed.

II. BACKGROUND AND PRELIMINARIES

A synthesizer can produce sounds that emulate musical instruments through different signals. To produce sound, synthesizers incorporate frequency oscillators, filters, low frequency oscillators, and ADSR envelope. The oscillator can generate different waveforms such as sine, sawtooth, square, or pulse waves. Filters can shape the frequency of the waves that get generated by either softening the high frequencies or removing lower frequencies. The low frequency oscillator generates effects that change the signal.

In our initial pitch for the simple synthesizer project, we wanted to incorporate choices for "synthesizer voice" by way of generating square waves, triangle waves, and sine waves. In addition, we wanted to create an ADSR envelope to give our FPGA instrument a more refined, pleasant sound. Unfortunately, we found no examples in our research of an ADSR envelope being used for an FPGA-based synthesizer, nor could we find FPGA sound examples that didn't use square waves. We chose to simplify our project further, our main focus becoming note generation at button press and changing pitch with an onboard switch.

Figure 1. Simple Synthesizer Functionality Flowchart

Since there was not much guidance for these concepts in previous classes, additional research was conducted to explore audio output methods to produce sounds on the FPGA. External sources guided team members in understanding the operations of the mono output port which combines the left and right audio channels into a single channel for sufficiency. Through this, basic sound tests were conducted , specifically from external resources, to validate its functionality [15].

Becoming familiar with the Nexys A7-100T's crystal oscillator and clock frequencies was a requirement for this project. Research was also conducted to find and draw inspiration from various implementations of a synthesizer to an FPGA. However, we heavily referenced two projects to attain specific formulas and techniques that are necessary for generating individual tones [5][6].

III. IMPLEMENTATION

A. *GET NOTES*

This module concerns the 37 possible notes that may play based on the signal retrieved from button presses. Likewise, the code for "pitch control" or "octave control" is in this module. Based on note frequency, the notes C3 to C6

will each have a corresponding counter register, flip-flop, and decimal number for flip-flop reset value. To be more specific, each note has a 20-bit register for its counter and each note has a corresponding bit in a 37-bit register for its flip-flop. To help create the tone, we can first calculate the number of clock pulses per cycle of a note with the equation $(1/frequency)\ *\ 10^6$ where frequency is the note's frequency in Hz [5].

Note	Freq (Hz)	Clock Pulses per Cycle	Note	Freq (Hz)	Clk Pulses per Cycle	Note	Freq (Hz)	Clock Pulses per Cycle
C3	130.81	7644	C4	261.63	3822	C5	523.25	1911
C3#	138.59	7215	C4#	277.18	3608	C5#	554.37	1804
D3	146.83	6811	D4	293.66	3405	D5	587.33	1703
D3#	155.56	6428	D4#	311.13	3214	D5#	622.25	1607
E3	164.81	6068	E4	329.63	3034	E5	659.25	1517
F3	174.61	5727	F4	349.23	2863	F5	698.46	1432
F3#	185.0	5405	F4#	369.99	2703	F5#	739.99	1351
G3	196.0	5102	G4	392.0	2551	G5	783.99	1276
G3#	207.65	4816	G4#	415.30	2408	G5#	830.61	1204
A3	220.0	4545	A4	440.0	2273	A5	880.0	1136
A3#	233.08	4290	A4#	466.16	2145	A5#	932.33	1073
B3	246.94	4050	B4	493.88	2025	B5	987.77	1012

Figure 2. Note Frequencies with Clock Pulses per Cycle

Each note's flip-flop reset value is found similarly with the equation

$$resetValue_{note} = 50\ *\ [(1/frequency)\ *\ 10^4]$$

When a button is pressed, the note's counter is equal to $50\ *\ resetValue_{note}$, and depending on the current octave setting (dictated by the 2-bit input for octave selection), its counter will be reset and its flip-flop will be toggled. The flip-flop toggles for as long as the button is pressed. Otherwise, the note's counter will count up until it reaches half of its tone's frequency. This process generates a square wave of a tone that will be expressed in the speaker output. If a key is not pressed, its flip-flop is reset or kept at "0" until pressed. An input for resetting the synthesizer is also included, which simply sets all note flip-flops to 0.

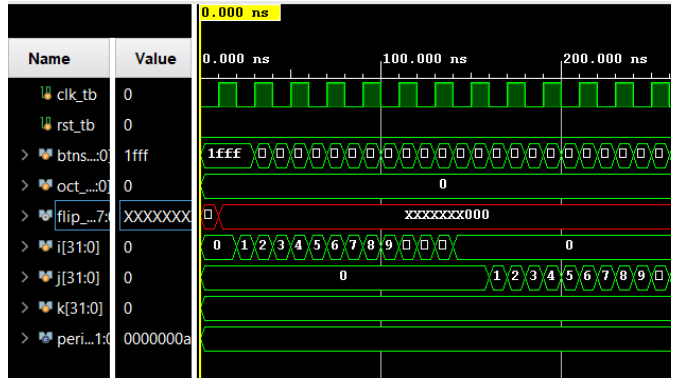

Figure 3a. Simulation of Get Notes Module

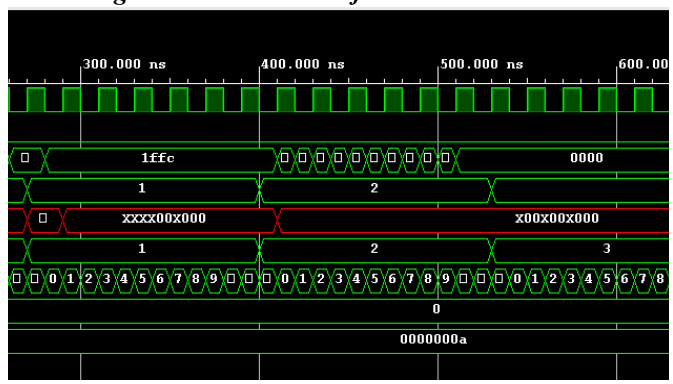

Figure 3b. Simulation of Get Notes Module

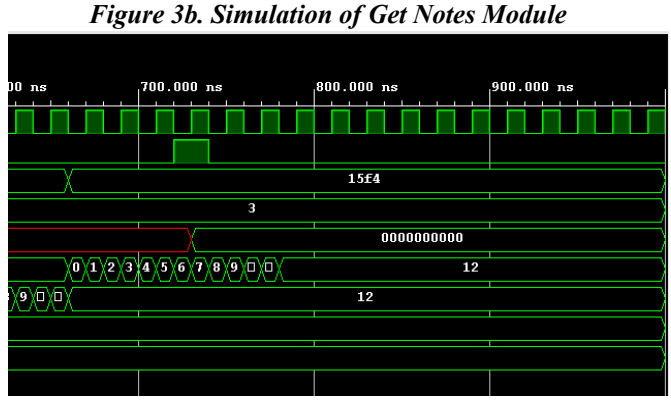

Figure 3c. Simulation of Get Notes Module

"Cut into parts to enlarge image"

The simulation for the Get Notes module revealed a discrepancy between the expected output value for the speaker and the actual tones produced when programming the board and testing it with the hardware. This divergence might occur due to limitations in the simulation environment. Simulations often simplify or abstract certain hardware behaviors, and consequently, the precise speaker output might not be accurately represented in the simulation.

B. *MAKE TONE*

This module concerns the speaker output. In MakeTone, there is a 37-bit input for the note-generation flip-flops and a single-bit output for the speaker. The output is assigned the Boolean sum of every available note flipper, or the bitwise OR of each register in the 37-bit input. This allows for multiple notes to be played at once.

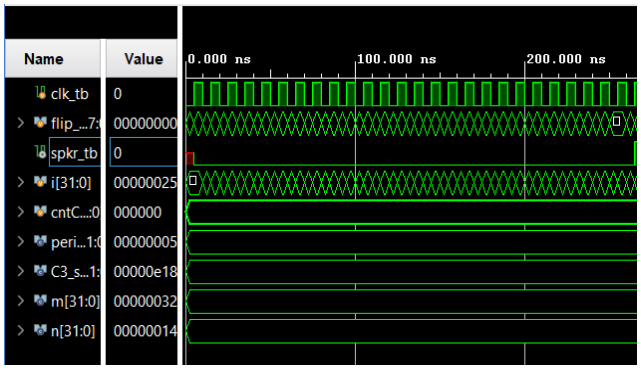

Figure 4a. Simulation of Make Tone Module

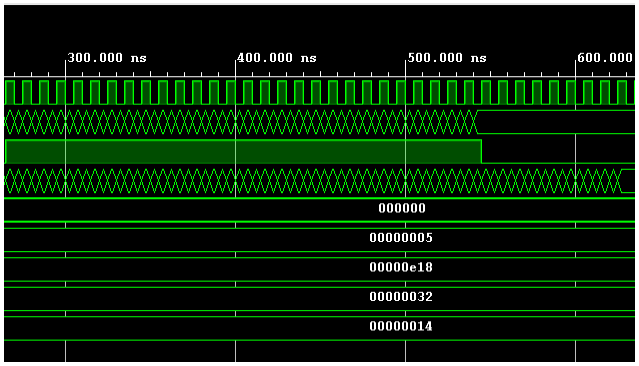

Figure 4a. Simulation of Make Tone Module

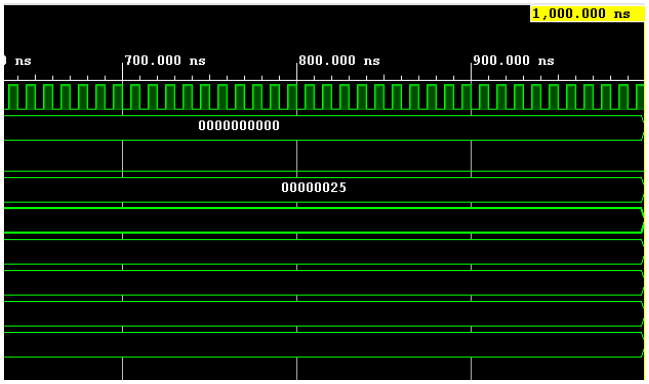

Figure 4a. Simulation of Make Tone Module

"Cut into parts to enlarge image"

C. *DISPLAY LIGHT*

This module associates buttons or key presses with specific onboard LEDs. A 12-bit output variable named LEDs is simply assigned with the inversion of the 12-bit input button variable.

D. *TOP MODULE*

The top module instantiates all other modules and connects them accordingly. Each module is connected to a clock. A wire is created for the flip-flops that concern wave generation. This wire is connected to the output of GetNotes and the input of MakeTone. The 13-bit variable for buttons is connected to the input of GetNotes and DisplayLight.

Figure 5a. Simulation of Top_tb with LED values showing

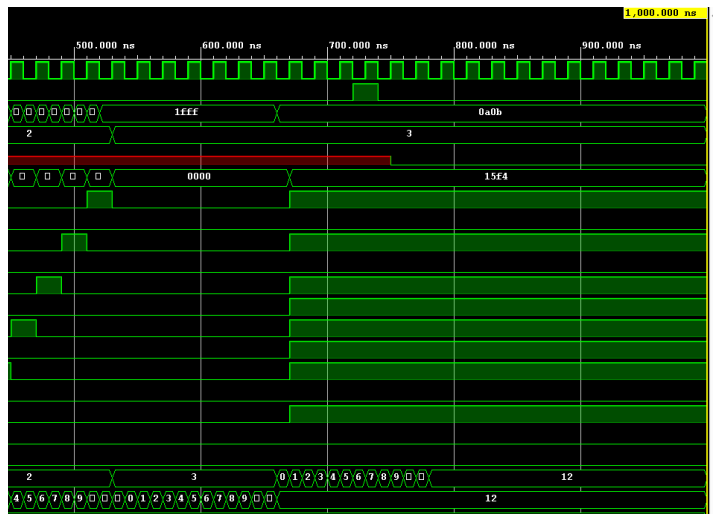

Figure 5b. Simulation of Top_tb with LED values showing

"Cut into parts to enlarge image"

IV. EVALUATION

Implementing this simple synthesizer was far more complex than we expected. It took several iterations of the GetNotes module and copious experimentation with waveform generation to reach our intended outcome. Though we heavily relied on other projects to reach our chosen technique for creating different tones with our FPGA, we struggled to grasp why certain equations and methods worked for us fully.

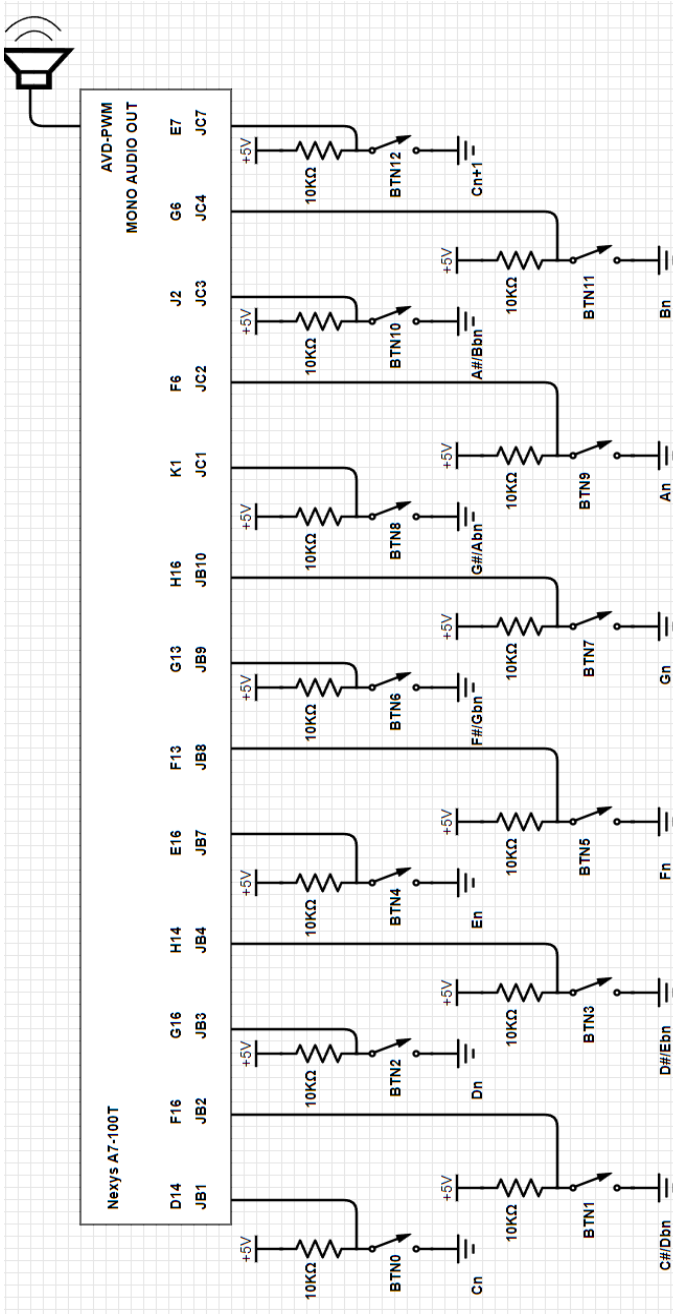

Figure 5. Peripherals Schematic

"Rotated to enlarge image"

In addition, testing and verifying our design, specifically creating simulations for the GetNotes module, wasn't ideal. In our test benches, analyzing the flip-flop register outputs would only yield "don't care"s when the outputs were expected to be visible. Despite this, the design would function as expected when programmed to our hardware. We considered alternative methods for testing our design. One approach would've been to use an FPGA board or a dedicated audio analyzer to measure and verify the tones produced by the synthesizer directly. These methods directly connect the hardware to an external measuring tool, bypassing simulation problems and providing a more accurate assessment of the speaker output. It's common for simulations to have limitations, especially concerning real-world analog outputs like audio outputs. Otherwise, using the FPGA's onboard LEDs and implementing octave switching was straightforward.

V. CONCLUSION

In conclusion, we successfully implemented a simple synthesizer that uses 12 buttons with 3 octaves through hardware. This was accomplished using the getNotes, makeTone, displayLights, and topSynth modules. The getNotes module contains the different notes that play with their respective frequencies. The makeTone module looks at what is being generated from the getNotes module and then takes that information and outputs the sound into the mono audio output. The displayLights module shows the corresponding buttons that are being pressed and shows an LED above each of the button presses on the Nexys Board. Finally, the topSynth module implements the functionalities of the other modules to create the simple synthesizer. With this implementation, we can generate different notes that can be played simultaneously or individually. While the implementation of the hardware was successful, we were not able to create a simulation testbench to check our note generation. The only method we used to verify our implementation was to compare the audio output of our synthesizer to an actual piano and with a tuner.

REFERENCES

[1] Digilent Inc., "Nexys A7: FPGA Trainer Board Reference Manual," Digilent Learn, 2018. [Online]. Available: https://learn.digilentinc.com/Documents/400. [Accessed: October 20, 2023].

[2] Instructables, "DIGITAL PIANO FPGA BOARD," 2019. [Online]. Available: https://www.instructables.com/DIGITAL-PIANO-FPGA-BOARD/. [Accessed: October 20, 2023].

[3] Columbia University, "EE3082: Digital Systems Fundamentals - Piano Synthesizer," lesson 5. [Online]. Available: https://www.bioee.ee.columbia.edu/courses/ee3082/piano/lesson5.html. [Accessed: October 20, 2023].

[4] Llamocca, L., et al., "FPGA Tone Generator for Digital Piano," Department of Electrical and Computer Engineering, Oakland University, Winter 2022, Final Project Group 2. [Online]. Available: https://www.secs.oakland.edu/~llamocca/Courses/ECE4710/W22/FinalProject/Group2_tonegen.pdf. [Accessed: October 20, 2023].

[5] ClockWorkKid, "FPGA-Based Piano." [Online]. Available: https://github.com/ClockWorkKid/FGPA-Based-Piano. [Accessed: October 20, 2023].

[6] Del Rosario, M., "CA Final Project." [Online]. Available: https://github.com/mdelrosa/cafinalproject. [Accessed: October 20, 2023].

[7] YouTube, "Poor Man's Piano." [Online]. Available: https://www.youtube.com/watch?v=95gMFHwd9gE. [Accessed: October 20, 2023].

[8] Digilent Inc., "DMA Audio Demo." [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-a7/demos/dma-audio. [Accessed: October 20, 2023].

[9] Min-Ho Hyun, Jong-Hyun Na and Sun-Young Hwang, "Design of a pipelined music synthesizer based on the wavetable method," in IEEE Transactions on Consumer Electronics, vol. 43, no. 3, pp. 605-613, Aug. 1997, doi: 10.1109/30.628683.

[10] R. Min, W. Yan-Nan, M. Guo-Qiang and Q. De-Hui, "The Electronic Piano Design Based on FPGA and PS2 Interface," 2013 Third International Conference on Intelligent System Design and Engineering Applications, Hong Kong, China, 2013, pp. 1442-1444, doi: 10.1109/ISDEA.2012.344.

[11] T. Shanthi and V. Krishnamurthi, "FPGA based frequency synthesizer for 14-band MB-OFDM UWB transceivers," 2016 International Conference on Emerging Trends in Engineering, Technology and Science (ICETETS), Pudukkottai, India, 2016, pp. 1-4, doi: 10.1109/ICETETS.2016.7603072.

[12] K. Balasubramanian, "High performance digital Music synthesizer," in IEEE Transactions on Consumer Electronics, vol. 38, no. 2, pp. 85-90, May 1992, doi: 10.1109/30.142863.

[13] M.A. Asyraf Khairuddin et al., "Digital Piano Keyboard Design using FPGA Implementation for Beginner," IOP Conference Series: Materials Science and Engineering, vol. 917, no. 1, p. 012053, sep 2020, doi: 10.1088/1757-899X/917/1/012053.

[14] R. Kusumah, D. Cahyadi, G. Kumara and T. Adiono, "CORDIC-based digital sound synthesizer," TENCON 2015 - 2015 IEEE Region 10 Conference, Macao, China, 2015, pp. 1-6, doi: 10.1109/TENCON.2015.7373158.

[15] FPGA4Fun, "Music Box." [Online]. Available: https://www.fpga4fun.com/MusicBox.html. [Accessed: October 20, 2023].

[16] Produce Like A Pro, "What Is a Synthesizer?" [Online]. Available: https://producelikeapro.com/blog/what-is-a-synthesizer/. [Accessed: October 20, 2023].